JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.stream

Interface IntStream

All Superinterfaces:

AutoCloseable, BaseStream<Integer,IntStream>

```
public interface IntStream
extends BaseStream<Integer,IntStream>
```

A sequence of primitive int-valued elements supporting sequential and parallel aggregate operations. This is the `int` primitive specialization of `Stream`.

The following example illustrates an aggregate operation using `Stream` and `IntStream`, computing the sum of the weights of the red widgets:

```
int sum = widgets.stream()
                 .filter(w -> w.getColor() == RED)
                 .mapToInt(w -> w.getWeight())
                 .sum();
```

See the class documentation for `Stream` and the package documentation for java.util.stream for additional specification of streams, stream operations, stream pipelines, and parallelism.

Since:

1.8

See Also:

Stream, java.util.stream

Nested Class Summary

Nested Classes	Interface	Description
static interface	IntStream.Builder	A mutable builder for an IntStream.

Method Summary

Default Methods | Abstract M

Modifier and Type	Method	Description
boolean	allMatch(IntPredicate predicate)	Returns whether elements of this s match the provid predicate.
boolean	anyMatch(IntPredicate predicate)	Returns whether elements of this s match the provid predicate.
DoubleStream	asDoubleStream()	Returns a DoubleStream consisting of the elements of this stream, converte double.
LongStream	asLongStream()	Returns a LongSt consisting of the elements of this stream, converte long.
OptionalDouble	average()	Returns an OptionalDouble describing the arithmetic mean elements of this stream, or an em optional if this st is empty.
Stream<Integer>	boxed()	Returns a Strea consisting of the elements of this stream, each box an Integer.

static **IntStream.Builder**	**builder**()	Returns a builde an IntStream.
<R> R	**collect**(**Supplier**<R> supplier, **ObjIntConsumer**<R> accumulator, **BiConsumer**<R,R> combiner)	Performs a **muta reduction** opera on the elements stream.
static **IntStream**	**concat**(**IntStream** a, **IntStream** b)	Creates a lazily concatenated str whose elements the elements of t first stream follo by all the elemen the second strea
long	**count**()	Returns the coun elements in this stream.
IntStream	**distinct**()	Returns a stream consisting of the distinct elements this stream.
static **IntStream**	**empty**()	Returns an empt sequential IntSt
IntStream	**filter**(**IntPredicate** predicate)	Returns a stream consisting of the elements of this that match the g predicate.
OptionalInt	**findAny**()	Returns an **OptionalInt** describing some element of the st or an empty OptionalInt if t stream is empty.
OptionalInt	**findFirst**()	Returns an **OptionalInt** describing the fi element of this st or an empty OptionalInt if t stream is empty.

IntStream	**flatMap**(**IntFunction**<? extends **IntStream**> mapper)	Returns a stream consisting of the results of replaci each element of t stream with the contents of a ma stream produced applying the prov mapping functio each element.
void	**forEach**(**IntConsumer** action)	Performs an acti each element of t stream.
void	**forEachOrdered**(**IntConsumer** action)	Performs an acti each element of t stream, guarante that each elemen processed in enc order for streams have a defined encounter order.
static **IntStream**	**generate**(**IntSupplier** s)	Returns an infini sequential unord stream where ea element is gener by the provided `IntSupplier`.
static **IntStream**	**iterate**(int seed, **IntUnaryOperator** f)	Returns an infini sequential ordere `IntStream` produ by iterative appli of a function `f` to initial element `se` producing a `Stre` consisting of `see` `f(seed)`, `f(f(se` etc.
PrimitiveIterator.OfInt	**iterator**()	Returns an iterat the elements of t stream.
IntStream	**limit**(long maxSize)	Returns a stream consisting of the elements of this

		stream, truncate no longer than ma in length.
IntStream	**map**(**IntUnaryOperator** mapper)	Returns a stream consisting of the results of applyin given function to elements of this stream.
DoubleStream	**mapToDouble**(**IntToDoubleFunction** mapper)	Returns a DoubleStream consisting of the results of applyin given function to elements of this stream.
LongStream	**mapToLong**(**IntToLongFunction** mapper)	Returns a LongSt consisting of the results of applyin given function to elements of this stream.
<U> **Stream**<U>	**mapToObj**(**IntFunction**<? extends U> mapper)	Returns an objec valued Stream consisting of the results of applyin given function to elements of this stream.
OptionalInt	**max**()	Returns an OptionalInt describing the maximum elemen this stream, or a empty optional if stream is empty.
OptionalInt	**min**()	Returns an OptionalInt describing the minimum elemen this stream, or a empty optional if stream is empty.

boolean	**noneMatch**(**IntPredicate** predicate)	Returns whether elements of this match the provid predicate.
static **IntStream**	**of**(int... values)	Returns a sequer ordered stream v elements are the specified values.
static **IntStream**	**of**(int t)	Returns a sequer `IntStream` conta single element.
IntStream	**parallel**()	Returns an equiv stream that is pa
IntStream	**peek**(**IntConsumer** action)	Returns a stream consisting of the elements of this stream, additiona performing the provided action o each element as elements are consumed from t resulting stream.
static **IntStream**	**range**(int startInclusive, int endExclusive)	Returns a sequer ordered `IntStre` from `startInclu` (inclusive) to `endExclusive` (exclusive) by an incremental step
static **IntStream**	**rangeClosed**(int startInclusive, int endInclusive)	Returns a sequer ordered `IntStre` from `startInclu` (inclusive) to `endInclusive` (inclusive) by an incremental step
OptionalInt	**reduce**(**IntBinaryOperator** op)	Performs a **redu** on the elements stream, using an **associative** accumulation fur

		and returns an `OptionalInt` describing the re value, if any.
`int`	**`reduce`**`(int identity, `**`IntBinaryOperator`**` op)`	Performs a **redu** on the elements stream, using the provided identity and an **associati** accumulation fun and returns the reduced value.
`IntStream`	**`sequential`**`()`	Returns an equiv stream that is sequential.
`IntStream`	**`skip`**`(long n)`	Returns a stream consisting of the remaining eleme this stream after discarding the fi elements of the s
`IntStream`	**`sorted`**`()`	Returns a stream consisting of the elements of this in sorted order.
`Spliterator.OfInt`	**`spliterator`**`()`	Returns a spliter for the elements stream.
`int`	**`sum`**`()`	Returns the sum elements in this stream.
`IntSummaryStatistics`	**`summaryStatistics`**`()`	Returns an `IntSummaryStat` describing variou summary data ab the elements of t stream.
`int[]`	**`toArray`**`()`	Returns an array containing the elements of this stream.

Methods inherited from interface java.util.stream.BaseStream

close, isParallel, onClose, unordered

Method Detail

filter

IntStream filter(IntPredicate predicate)

Returns a stream consisting of the elements of this stream that match the given predicate.

This is an intermediate operation.

Parameters:

predicate - a non-interfering, stateless predicate to apply to each element to determine if it should be included

Returns:

the new stream

map

IntStream map(IntUnaryOperator mapper)

Returns a stream consisting of the results of applying the given function to the elements of this stream.

This is an intermediate operation.

Parameters:

mapper - a non-interfering, stateless function to apply to each element

Returns:

the new stream

mapToObj

<U> Stream<U> mapToObj(IntFunction<? extends U> mapper)

Returns an object-valued Stream consisting of the results of applying the given function to the elements of this stream.

This is an intermediate operation.

Type Parameters:

`U - the element type of the new stream`

Parameters:

`mapper - a non-interfering, stateless function to apply to each element`

Returns:

`the new stream`

mapToLong

`LongStream mapToLong(IntToLongFunction mapper)`

Returns a `LongStream` consisting of the results of applying the given function to the elements of this stream.

This is an intermediate operation.

Parameters:

`mapper - a non-interfering, stateless function to apply to each element`

Returns:

`the new stream`

mapToDouble

`DoubleStream mapToDouble(IntToDoubleFunction mapper)`

Returns a `DoubleStream` consisting of the results of applying the given function to the elements of this stream.

This is an intermediate operation.

Parameters:

`mapper - a non-interfering, stateless function to apply to each element`

Returns:

`the new stream`

flatMap

`IntStream flatMap(IntFunction<? extends IntStream> mapper)`

Returns a stream consisting of the results of replacing each element of this stream with the contents of a mapped stream produced by applying the provided mapping function to each element. Each mapped stream is `closed` after its contents have been placed into this stream. (If a mapped stream is `null` an empty stream is used, instead.)

This is an intermediate operation.

Parameters:

`mapper - a non-interfering, stateless function to apply to each element which produces an IntStream of new values`

Returns:

`the new stream`

See Also:

`Stream.flatMap(Function)`

distinct

`IntStream distinct()`

Returns a stream consisting of the distinct elements of this stream.

This is a stateful intermediate operation.

Returns:

`the new stream`

sorted

`IntStream sorted()`

Returns a stream consisting of the elements of this stream in sorted order.

This is a stateful intermediate operation.

Returns:

`the new stream`

peek

`IntStream peek(IntConsumer action)`

Returns a stream consisting of the elements of this stream, additionally performing the provided action on each element as elements are consumed from the resulting stream.

This is an intermediate operation.

For parallel stream pipelines, the action may be called at whatever time and in whatever thread the element is made available by the upstream operation. If the action modifies shared state, it is responsible for providing the required synchronization.

API Note:

This method exists mainly to support debugging, where you want to see the elements as they flow past a certain point in a pipeline:

```
IntStream.of(1, 2, 3, 4)
    .filter(e -> e > 2)
    .peek(e -> System.out.println("Filtered value: " + e))
    .map(e -> e * e)
    .peek(e -> System.out.println("Mapped value: " + e))
    .sum();
```

Parameters:

`action` - a non-interfering action to perform on the elements as they are consumed from the stream

Returns:

the new stream

limit

`IntStream limit(long maxSize)`

Returns a stream consisting of the elements of this stream, truncated to be no longer than `maxSize` in length.

This is a short-circuiting stateful intermediate operation.

API Note:

While limit() is generally a cheap operation on sequential stream pipelines, it can be quite expensive on ordered parallel pipelines, especially for large values of maxSize, since limit(n) is constrained to return not just any *n* elements, but the *first n* elements in the encounter order. Using an unordered stream source (such as generate(IntSupplier)) or removing the ordering constraint with BaseStream.unordered() may result in significant speedups of limit() in parallel pipelines, if the semantics of your situation permit. If consistency with encounter order is required, and you are experiencing poor performance or memory utilization with limit() in parallel pipelines, switching to sequential execution with sequential() may improve performance.

Parameters:

maxSize - the number of elements the stream should be limited to

Returns:

the new stream

Throws:

IllegalArgumentException - if maxSize is negative

skip

IntStream skip(long n)

Returns a stream consisting of the remaining elements of this stream after discarding the first n elements of the stream. If this stream contains fewer than n elements then an empty stream will be returned.

This is a stateful intermediate operation.

API Note:

While skip() is generally a cheap operation on sequential stream pipelines, it can be quite expensive on ordered parallel pipelines, especially for large values of n, since skip(n) is constrained to skip not just any *n* elements, but the *first n* elements in the encounter order. Using an unordered stream source (such as generate(IntSupplier)) or removing the ordering constraint with BaseStream.unordered() may result in significant speedups of skip() in parallel pipelines, if the semantics of your situation permit. If consistency with encounter order is required, and you are experiencing poor performance or memory utilization with skip() in parallel pipelines, switching to sequential execution with sequential() may improve performance.

Parameters:

n - the number of leading elements to skip

Returns:

the new stream

Throws:

IllegalArgumentException - if n is negative

forEach

void forEach(IntConsumer action)

Performs an action for each element of this stream.

This is a terminal operation.

For parallel stream pipelines, this operation does *not* guarantee to respect the encounter order of the stream, as doing so would sacrifice the benefit of parallelism. For any given element, the action may be performed at whatever time and in whatever thread the library chooses. If the action accesses shared state, it is responsible for providing the required synchronization.

Parameters:

`action - a non-interfering action to perform on the elements`

forEachOrdered

```
void forEachOrdered(IntConsumer action)
```

Performs an action for each element of this stream, guaranteeing that each element is processed in encounter order for streams that have a defined encounter order.

This is a terminal operation.

Parameters:

`action - a non-interfering action to perform on the elements`

See Also:

`forEach(IntConsumer)`

toArray

```
int[] toArray()
```

Returns an array containing the elements of this stream.

This is a terminal operation.

Returns:

`an array containing the elements of this stream`

reduce

```
int reduce(int identity,
           IntBinaryOperator op)
```

Performs a reduction on the elements of this stream, using the provided identity value and an associative accumulation function, and returns the reduced value. This is equivalent to:

```
     int result = identity;
```

```
     for (int element : this stream)
         result = accumulator.applyAsInt(result, element)
     return result;
```

but is not constrained to execute sequentially.

The `identity` value must be an identity for the accumulator function. This means that for all `x`, `accumulator.apply(identity, x)` is equal to `x`. The `accumulator` function must be an associative function.

This is a terminal operation.

API Note:

Sum, min, max, and average are all special cases of reduction. Summing a stream of numbers can be expressed as:

```
     int sum = integers.reduce(0, (a, b) -> a+b);
```

or more compactly:

```
     int sum = integers.reduce(0, Integer::sum);
```

While this may seem a more roundabout way to perform an aggregation compared to simply mutating a running total in a loop, reduction operations parallelize more gracefully, without needing additional synchronization and with greatly reduced risk of data races.

Parameters:

identity - the identity value for the accumulating function

op - an associative, non-interfering, stateless function for combining two values

Returns:

the result of the reduction

See Also:

sum(), min(), max(), average()

reduce

```
OptionalInt reduce(IntBinaryOperator op)
```

Performs a reduction on the elements of this stream, using an associative accumulation function, and returns an `OptionalInt` describing the reduced value, if any. This is equivalent to:

```
boolean foundAny = false;
int result = null;
for (int element : this stream) {
    if (!foundAny) {
        foundAny = true;
        result = element;
    }
    else
        result = accumulator.applyAsInt(result, element);
}
return foundAny ? OptionalInt.of(result) : OptionalInt.empty();
```

but is not constrained to execute sequentially.

The `accumulator` function must be an associative function.

This is a terminal operation.

Parameters:

`op - an associative, non-interfering, stateless function for combining two values`

Returns:

`the result of the reduction`

See Also:

`reduce(int, IntBinaryOperator)`

collect

```
<R> R collect(Supplier<R> supplier,
              ObjIntConsumer<R> accumulator,
              BiConsumer<R,R> combiner)
```

Performs a mutable reduction operation on the elements of this stream. A mutable reduction is one in which the reduced value is a mutable result container, such as an `ArrayList`, and elements are incorporated by updating the state of the result rather than by replacing the result. This produces a result equivalent to:

```
R result = supplier.get();
for (int element : this stream)
    accumulator.accept(result, element);
```

```
    return result;
```

Like `reduce(int, IntBinaryOperator)`, `collect` operations can be parallelized without requiring additional synchronization.

This is a terminal operation.

Type Parameters:

`R - type of the result`

Parameters:

`supplier - a function that creates a new result container. For a parallel execution, this function may be called multiple times and must return a fresh value each time.`

`accumulator - an associative, non-interfering, stateless function for incorporating an additional element into a result`

`combiner - an associative, non-interfering, stateless function for combining two values, which must be compatible with the accumulator function`

Returns:

`the result of the reduction`

See Also:

`Stream.collect(Supplier, BiConsumer, BiConsumer)`

sum

```
int sum()
```

Returns the sum of elements in this stream. This is a special case of a reduction and is equivalent to:

```
    return reduce(0, Integer::sum);
```

This is a terminal operation.

Returns:

`the sum of elements in this stream`

min

```
OptionalInt min()
```

Returns an `OptionalInt` describing the minimum element of this stream, or an empty optional if this stream is empty. This is a special case of a reduction and is equivalent to:

```
return reduce(Integer::min);
```

This is a terminal operation.

Returns:

an OptionalInt containing the minimum element of this stream, or an empty OptionalInt if the stream is empty

max

`OptionalInt max()`

Returns an `OptionalInt` describing the maximum element of this stream, or an empty optional if this stream is empty. This is a special case of a reduction and is equivalent to:

```
return reduce(Integer::max);
```

This is a terminal operation.

Returns:

an OptionalInt containing the maximum element of this stream, or an empty OptionalInt if the stream is empty

count

`long count()`

Returns the count of elements in this stream. This is a special case of a reduction and is equivalent to:

```
return mapToLong(e -> 1L).sum();
```

This is a terminal operation.

Returns:

the count of elements in this stream

average

`OptionalDouble average()`

Returns an `OptionalDouble` describing the arithmetic mean of elements of this stream, or an empty optional if this stream is empty. This is a special case of a reduction.

This is a terminal operation.

Returns:

an `OptionalDouble` containing the average element of this stream, or an empty optional if the stream is empty

summaryStatistics

`IntSummaryStatistics summaryStatistics()`

Returns an `IntSummaryStatistics` describing various summary data about the elements of this stream. This is a special case of a reduction.

This is a terminal operation.

Returns:

an `IntSummaryStatistics` describing various summary data about the elements of this stream

anyMatch

`boolean anyMatch(IntPredicate predicate)`

Returns whether any elements of this stream match the provided predicate. May not evaluate the predicate on all elements if not necessary for determining the result. If the stream is empty then `false` is returned and the predicate is not evaluated.

This is a short-circuiting terminal operation.

API Note:

This method evaluates the *existential quantification* of the predicate over the elements of the stream (for some x P(x)).

Parameters:

`predicate` - a non-interfering, stateless predicate to apply to elements of this stream

Returns:

true if any elements of the stream match the provided predicate, otherwise false

allMatch

boolean allMatch(IntPredicate predicate)

Returns whether all elements of this stream match the provided predicate. May not evaluate the predicate on all elements if not necessary for determining the result. If the stream is empty then true is returned and the predicate is not evaluated.

This is a short-circuiting terminal operation.

API Note:

This method evaluates the *universal quantification* of the predicate over the elements of the stream (for all x P(x)). If the stream is empty, the quantification is said to be *vacuously satisfied* and is always true (regardless of P(x)).

Parameters:

predicate - a non-interfering, stateless predicate to apply to elements of this stream

Returns:

true if either all elements of the stream match the provided predicate or the stream is empty, otherwise false

noneMatch

boolean noneMatch(IntPredicate predicate)

Returns whether no elements of this stream match the provided predicate. May not evaluate the predicate on all elements if not necessary for determining the result. If the stream is empty then true is returned and the predicate is not evaluated.

This is a short-circuiting terminal operation.

API Note:

This method evaluates the *universal quantification* of the negated predicate over the elements of the stream (for all x ~P(x)). If the stream is empty, the quantification is said to be vacuously satisfied and is always true, regardless of P(x).

Parameters:

predicate - a non-interfering, stateless predicate to apply to elements of this stream

Returns:

`true if either no elements of the stream match the provided predicate or the stream is empty, otherwise false`

findFirst

`OptionalInt findFirst()`

Returns an `OptionalInt` describing the first element of this stream, or an empty `OptionalInt` if the stream is empty. If the stream has no encounter order, then any element may be returned.

This is a short-circuiting terminal operation.

Returns:

`an OptionalInt describing the first element of this stream, or an empty OptionalInt if the stream is empty`

findAny

`OptionalInt findAny()`

Returns an `OptionalInt` describing some element of the stream, or an empty `OptionalInt` if the stream is empty.

This is a short-circuiting terminal operation.

The behavior of this operation is explicitly nondeterministic; it is free to select any element in the stream. This is to allow for maximal performance in parallel operations; the cost is that multiple invocations on the same source may not return the same result. (If a stable result is desired, use `findFirst()` instead.)

Returns:

`an OptionalInt describing some element of this stream, or an empty OptionalInt if the stream is empty`

See Also:

`findFirst()`

asLongStream

`LongStream asLongStream()`

Returns a `LongStream` consisting of the elements of this stream, converted to `long`.

This is an intermediate operation.

Returns:

`a LongStream consisting of the elements of this stream, converted to long`

asDoubleStream

```
DoubleStream asDoubleStream()
```

Returns a `DoubleStream` consisting of the elements of this stream, converted to `double`.

This is an intermediate operation.

Returns:

`a DoubleStream consisting of the elements of this stream, converted to double`

boxed

```
Stream<Integer> boxed()
```

Returns a `Stream` consisting of the elements of this stream, each boxed to an `Integer`.

This is an intermediate operation.

Returns:

`a Stream consistent of the elements of this stream, each boxed to an Integer`

sequential

```
IntStream sequential()
```

Description copied from interface: `BaseStream`

Returns an equivalent stream that is sequential. May return itself, either because the stream was already sequential, or because the underlying stream state was modified to be sequential.

This is an intermediate operation.

Specified by:

`sequential` in interface `BaseStream<Integer,IntStream>`

Returns:

a sequential stream

parallel

`IntStream parallel()`

Description copied from interface: BaseStream

Returns an equivalent stream that is parallel. May return itself, either because the stream was already parallel, or because the underlying stream state was modified to be parallel.

This is an intermediate operation.

Specified by:

`parallel in interface BaseStream<Integer,IntStream>`

Returns:

a parallel stream

iterator

`PrimitiveIterator.OfInt iterator()`

Description copied from interface: BaseStream

Returns an iterator for the elements of this stream.

This is a terminal operation.

Specified by:

`iterator in interface BaseStream<Integer,IntStream>`

Returns:

the element iterator for this stream

spliterator

`Spliterator.OfInt spliterator()`

Description copied from interface: BaseStream
Returns a spliterator for the elements of this stream.

This is a terminal operation.

Specified by:
`spliterator in interface BaseStream<Integer,IntStream>`

Returns:
the element spliterator for this stream

builder

`static IntStream.Builder builder()`

Returns a builder for an `IntStream`.

Returns:
a stream builder

empty

`static IntStream empty()`

Returns an empty sequential `IntStream`.

Returns:
an empty sequential stream

of

`static IntStream of(int t)`

Returns a sequential `IntStream` containing a single element.

Parameters:
t - the single element

Returns:
a singleton sequential stream

of

```
static IntStream of(int... values)
```

Returns a sequential ordered stream whose elements are the specified values.

Parameters:

`values - the elements of the new stream`

Returns:

`the new stream`

iterate

```
static IntStream iterate(int seed,
                         IntUnaryOperator f)
```

Returns an infinite sequential ordered `IntStream` produced by iterative application of a function `f` to an initial element `seed`, producing a `Stream` consisting of `seed`, `f(seed)`, `f(f(seed))`, etc.

The first element (position `0`) in the `IntStream` will be the provided `seed`. For `n > 0`, the element at position `n`, will be the result of applying the function `f` to the element at position `n - 1`.

Parameters:

`seed - the initial element`

`f - a function to be applied to the previous element to produce a new element`

Returns:

`A new sequential IntStream`

generate

```
static IntStream generate(IntSupplier s)
```

Returns an infinite sequential unordered stream where each element is generated by the provided `IntSupplier`. This is suitable for generating constant streams, streams of random elements, etc.

Parameters:

`s - the IntSupplier for generated elements`

Returns:

`a new infinite sequential unordered IntStream`

range

```
static IntStream range(int startInclusive,
                       int endExclusive)
```

Returns a sequential ordered `IntStream` from `startInclusive` (inclusive) to `endExclusive` (exclusive) by an incremental step of 1.

API Note:

An equivalent sequence of increasing values can be produced sequentially using a for loop as follows:

```
for (int i = startInclusive; i < endExclusive ; i++) { ... }
```

Parameters:

`startInclusive` - the (inclusive) initial value

`endExclusive` - the exclusive upper bound

Returns:

a sequential IntStream for the range of int elements

rangeClosed

```
static IntStream rangeClosed(int startInclusive,
                             int endInclusive)
```

Returns a sequential ordered `IntStream` from `startInclusive` (inclusive) to `endInclusive` (inclusive) by an incremental step of 1.

API Note:

An equivalent sequence of increasing values can be produced sequentially using a for loop as follows:

```
for (int i = startInclusive; i <= endInclusive ; i++) { ... }
```

Parameters:

`startInclusive` - the (inclusive) initial value

`endInclusive` - the inclusive upper bound

Returns:

a sequential IntStream for the range of int elements

concat

```
static IntStream concat(IntStream a,
                        IntStream b)
```

Creates a lazily concatenated stream whose elements are all the elements of the first stream followed by all the elements of the second stream. The resulting stream is ordered if both of the input streams are ordered, and parallel if either of the input streams is parallel. When the resulting stream is closed, the close handlers for both input streams are invoked.

Implementation Note:

Use caution when constructing streams from repeated concatenation. Accessing an element of a deeply concatenated stream can result in deep call chains, or even StackOverflowException.

Parameters:

a - the first stream

b - the second stream

Returns:

the concatenation of the two input streams

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.